

PYTHON PROGRAMMING

COMPLETE HANDBOOK — BASIC TO ADVANCED

PART 9

Module 25: Projects

Beginner · Intermediate · Advanced

Continues from Parts 1-8 (Modules 1-24)

TABLE OF CONTENTS

TABLE OF CONTENTS	2
MODULE 25: PROJECTS	3
BEGINNER PROJECTS	4
Project 1: Calculator.....	4
Project 2: Number Guessing Game	6
Project 3: ATM System.....	7
Project 4: Student Management System	9
INTERMEDIATE PROJECTS	11
Project 5: Library Management System.....	11
Project 6: Employee Management System	13
Project 7: Expense Tracker	15
ADVANCED PROJECTS	17
Project 8: E-commerce Sales Analysis	17
Project 9: Face Detection.....	19
Project 10: Rule-Based Chatbot.....	20
Project 11: Crop Price Prediction	21
MORE PROJECT IDEAS FOR PRACTICE	22

MODULE 25: PROJECTS

This module brings together everything from earlier modules into ten complete, working projects — four beginner, three intermediate, and three advanced. Each project includes a problem statement, algorithm, a text-based flowchart, complete runnable code, sample output, and an explanation. Type out each program yourself rather than just reading it — that's how the concepts truly stick.

Project	Level	Key Concepts Used
1. Calculator	Beginner	Functions, if-elif, loops
2. Number Guessing Game	Beginner	random module, while loop
3. ATM System	Beginner	Classes, encapsulation
4. Student Management System	Beginner	Lists of dicts, CRUD, menu loop
5. Library Management System	Intermediate	OOP, dictionaries, file persistence
6. Employee Management System	Intermediate	CSV file handling, CRUD
7. Expense Tracker	Intermediate	JSON persistence, aggregation
8. E-commerce Sales Analysis	Advanced	Pandas, groupby, visualisation
9. Face Detection	Advanced	OpenCV, Haar cascades
10. Rule-Based Chatbot	Advanced	String matching, dictionaries, loops
11. Crop Price Prediction	Advanced	scikit-learn, Linear Regression

BEGINNER PROJECTS

Project 1: Calculator

Difficulty: Beginner

Problem Statement: Build a menu-driven calculator that performs addition, subtraction, multiplication, and division on two numbers entered by the user, repeating until they choose to exit.

Algorithm:

1. Display a menu of operations.
2. Take the user's operation choice.
3. Take two numbers as input.
4. Perform the chosen operation and display the result.
5. Ask if the user wants to continue; repeat or exit accordingly.

```
[Start] -> [Show Menu] -> [Get Choice]
      |
      v
[Get num1, num2] -> [Perform Operation] -> [Show Result]
      |
      v
[Continue? ]--Yes--> back to [Show Menu]
      |
      No
      v
[End]
```

```

def calculator():
    while True:
        print("\n--- CALCULATOR ---")
        print("1. Add  2. Subtract  3. Multiply  4. Divide  5. Exit")
        choice = input("Choose an option (1-5): ")

        if choice == "5":
            print("Goodbye!")
            break

        if choice not in ("1", "2", "3", "4"):
            print("Invalid choice, try again.")
            continue

        try:
            num1 = float(input("Enter first number: "))
            num2 = float(input("Enter second number: "))
        except ValueError:
            print("Please enter valid numbers.")
            continue

        if choice == "1":
            print("Result:", num1 + num2)
        elif choice == "2":
            print("Result:", num1 - num2)
        elif choice == "3":
            print("Result:", num1 * num2)
        elif choice == "4":
            if num2 == 0:
                print("Error: cannot divide by zero")
            else:
                print("Result:", num1 / num2)

calculator()

```

Output:

```

--- CALCULATOR ---
1. Add  2. Subtract  3. Multiply  4. Divide  5. Exit
Choose an option (1-5): 1
Enter first number: 10
Enter second number: 5
Result: 15.0

```

Explanation: The outer while True loop keeps the menu running until the user picks '5'. try-except catches invalid number input gracefully, and the division branch specifically checks for zero to avoid crashing with ZeroDivisionError.

Project 2: Number Guessing Game

Difficulty: Beginner

Problem Statement: The computer picks a random number between 1 and 100. The player guesses repeatedly, receiving 'Too high' or 'Too low' hints, until they guess correctly, and the program reports how many attempts it took.

Algorithm:

1. Generate a random target number between 1 and 100.
2. Repeat: ask the player for a guess.
3. Compare the guess to the target and give a hint (too high/too low/correct).
4. Count attempts; stop the loop once guessed correctly.

```
import random

def guessing_game():
    target = random.randint(1, 100)
    attempts = 0
    print("Guess a number between 1 and 100!")

    while True:
        guess = int(input("Your guess: "))
        attempts += 1
        if guess < target:
            print("Too low! Try again.")
        elif guess > target:
            print("Too high! Try again.")
        else:
            print(f"Correct! You guessed it in {attempts} attempts.")
            break

guessing_game()
```

Output:

```
Guess a number between 1 and 100!
Your guess: 50
Too high! Try again.
Your guess: 25
Too low! Try again.
Your guess: 37
Correct! You guessed it in 3 attempts.
```

Explanation: `random.randint(1, 100)` picks the secret number once, outside the loop. The `while True` loop keeps asking for guesses, using `if-elif-else` to give feedback, and `break` exits the loop the moment the guess matches the target.

Project 3: ATM System

Difficulty: Beginner

Problem Statement: Simulate a simple ATM using a class — the user must enter a correct PIN, then can check balance, deposit, or withdraw money through a menu, with the balance protected as a private attribute.

Algorithm:

1. Create an ATM class storing PIN and balance privately.
2. Verify the PIN before allowing any transaction.
3. Show a menu: check balance, deposit, withdraw, exit.
4. Validate withdrawals against the current balance.

```

class ATM:
    def __init__(self, pin, balance=0):
        self.__pin = pin
        self.__balance = balance

    def verify_pin(self, entered_pin):
        return entered_pin == self.__pin

    def check_balance(self):
        return self.__balance

    def deposit(self, amount):
        self.__balance += amount
        print(f"Deposited ₹{amount}. New balance: ₹{self.__balance}")

    def withdraw(self, amount):
        if amount > self.__balance:
            print("Insufficient balance!")
        else:
            self.__balance -= amount
            print(f"Withdrew ₹{amount}. New balance: ₹{self.__balance}")

my_atm = ATM(pin="1234", balance=5000)
entered = input("Enter PIN: ")

if my_atm.verify_pin(entered):
    while True:
        print("\n1. Check Balance  2. Deposit  3. Withdraw  4. Exit")
        choice = input("Choose option: ")
        if choice == "1":
            print("Balance: ₹" + str(my_atm.check_balance()))
        elif choice == "2":
            my_atm.deposit(float(input("Enter amount: ")))
        elif choice == "3":
            my_atm.withdraw(float(input("Enter amount: ")))
        elif choice == "4":
            print("Thank you for banking with us!")
            break
    else:
        print("Incorrect PIN!")

```

Output:

```

Enter PIN: 1234
1. Check Balance  2. Deposit  3. Withdraw  4. Exit
Choose option: 3
Enter amount: 1000
Withdrew ₹1000.0. New balance: ₹4000.0

```

Explanation: `__pin` and `__balance` are private attributes (encapsulation) — they cannot be accessed directly from outside the class, only through the class's own methods, protecting the account's sensitive data from accidental external modification.

Project 4: Student Management System

Difficulty: Beginner

Problem Statement: Build a menu-driven system to add, view, update, and delete student records (name and marks), stored in memory as a list of dictionaries.

Algorithm:

1. Store students as a list of dictionaries: {'name':..., 'marks':...}.
2. Menu options: Add, View All, Update, Delete, Exit.
3. For update/delete, search by student name first.

```

students = []

def add_student():
    name = input("Name: ")
    marks = float(input("Marks: "))
    students.append({"name": name, "marks": marks})
    print("Student added!")

def view_students():
    if not students:
        print("No records found.")
    for s in students:
        print(f"{s['name']} - {s['marks']}")

def update_student():
    name = input("Enter name to update: ")
    for s in students:
        if s["name"] == name:
            s["marks"] = float(input("Enter new marks: "))
            print("Updated!")
            return
    print("Student not found.")

def delete_student():
    name = input("Enter name to delete: ")
    for s in students:
        if s["name"] == name:
            students.remove(s)
            print("Deleted!")
            return
    print("Student not found.")

while True:
    print("\n1.Add 2.View 3.Update 4.Delete 5.Exit")
    ch = input("Choice: ")
    if ch == "1": add_student()
    elif ch == "2": view_students()
    elif ch == "3": update_student()
    elif ch == "4": delete_student()
    elif ch == "5": break

```

Output:

```

1.Add 2.View 3.Update 4.Delete 5.Exit
Choice: 1
Name: Riya
Marks: 88
Student added!

```

Explanation: Each student is a dictionary inside the students list. Update and delete both search linearly for a matching name field, demonstrating a simple in-memory CRUD pattern that mirrors how database-driven applications work.

INTERMEDIATE PROJECTS

Project 5: Library Management System

Difficulty: Intermediate

Problem Statement: Manage a small library's books using a class-based system that supports issuing and returning books, tracking availability.

Algorithm:

1. Represent the library as a dictionary of {title: {'copies':n, 'issued':m}}.
2. `issue_book()` reduces available copies if any remain.
3. `return_book()` increases available copies back.

```

class Library:
    def __init__(self):
        self.books = {}

    def add_book(self, title, copies):
        if title in self.books:
            self.books[title]["copies"] += copies
        else:
            self.books[title] = {"copies": copies, "issued": 0}
        print(f"Added {copies} copies of '{title}'")

    def issue_book(self, title):
        book = self.books.get(title)
        if not book or book["copies"] - book["issued"] <= 0:
            print("No copies available!")
            return
        book["issued"] += 1
        print(f"'{title}' issued successfully.")

    def return_book(self, title):
        book = self.books.get(title)
        if book and book["issued"] > 0:
            book["issued"] -= 1
            print(f"'{title}' returned successfully.")
        else:
            print("Invalid return.")

    def show_status(self):
        for title, info in self.books.items():
            available = info["copies"] - info["issued"]
            print(f"{title}: {available}/{info['copies']} available")

lib = Library()
lib.add_book("Python Basics", 3)
lib.issue_book("Python Basics")
lib.issue_book("Python Basics")
lib.show_status()
lib.return_book("Python Basics")
lib.show_status()

```

Output:

```

Added 3 copies of 'Python Basics'
'Python Basics' issued successfully.
'Python Basics' issued successfully.
Python Basics: 1/3 available
'Python Basics' returned successfully.
Python Basics: 2/3 available

```

Explanation: Each book tracks total copies and how many are currently issued; available copies are always copies - issued. This models a realistic, simplified library system using nested dictionaries inside a class.

Project 6: Employee Management System

Difficulty: Intermediate

Problem Statement: Store employee records (ID, name, salary) persistently in a CSV file, supporting add and view-all operations that survive across program runs.

Algorithm:

1. Use Python's csv module to append new employee rows to a file.
2. Read and display all rows from the CSV whenever requested.

```
import csv, os

FILE = "employees.csv"

def add_employee(emp_id, name, salary):
    file_exists = os.path.isfile(FILE)
    with open(FILE, "a", newline="") as f:
        writer = csv.writer(f)
        if not file_exists:
            writer.writerow(["ID", "Name", "Salary"])
        writer.writerow([emp_id, name, salary])
    print("Employee added!")

def view_employees():
    if not os.path.isfile(FILE):
        print("No records yet.")
        return
    with open(FILE, "r") as f:
        reader = csv.reader(f)
        for row in reader:
            print(" | ".join(row))

def total_payroll():
    if not os.path.isfile(FILE):
        return 0
    with open(FILE, "r") as f:
        reader = csv.reader(f)
        next(reader, None) # skip header row
        return sum(float(row[2]) for row in reader)

add_employee(101, "Riya", 55000)
add_employee(102, "Aman", 48000)
view_employees()
print("Total payroll:", total_payroll())
```

Output:

```
Employee added!  
Employee added!  
ID | Name | Salary  
101 | Riya | 55000  
102 | Aman | 48000  
Total payroll: 103000.0
```

Explanation: Data is persisted using CSV file append mode ('a'), so records survive between separate runs of the program — this is a step up from Project 4, which lost all data when the program ended.

Project 7: Expense Tracker

Difficulty: Intermediate

Problem Statement: Track daily expenses by category, save them persistently as JSON, and generate a category-wise summary report.

Algorithm:

1. Load existing expenses from a JSON file at startup (or start with an empty list).
2. Add new expenses (amount, category, note) to the list.
3. Save the updated list back to the JSON file after every change.
4. Summarise total spending per category using a dictionary.

```
import json, os

FILE = "expenses.json"

def load_expenses():
    if os.path.isfile(FILE):
        with open(FILE, "r") as f:
            return json.load(f)
    return []

def save_expenses(expenses):
    with open(FILE, "w") as f:
        json.dump(expenses, f, indent=2)

def add_expense(expenses, amount, category, note=""):
    expenses.append({"amount": amount, "category": category, "note": note})
    save_expenses(expenses)
    print("Expense added!")

def summary(expenses):
    totals = {}
    for e in expenses:
        totals[e["category"]] = totals.get(e["category"], 0) + e["amount"]
    for category, total in totals.items():
        print(f"{category}: ₹{total}")

expenses = load_expenses()
add_expense(expenses, 250, "Food", "Lunch")
add_expense(expenses, 1200, "Travel", "Cab fare")
add_expense(expenses, 150, "Food", "Snacks")
summary(expenses)
```

Output:

```
Expense added!  
Expense added!  
Expense added!  
Food: ₹400  
Travel: ₹1200
```

Explanation: `totals.get(category, 0)` safely returns 0 for a category seen for the first time, then adds the new amount — a common accumulation pattern that avoids a `KeyError` on the first occurrence of each category.

ADVANCED PROJECTS

About These Projects

These four projects introduce real external libraries (pandas, OpenCV, scikit-learn) and simplified versions of industry techniques. Install the relevant library with pip before running each one. They are intentionally simplified to remain teachable within a single program.

Project 8: E-commerce Sales Analysis

Difficulty: Advanced

Problem Statement: Given a small sales dataset, find the best-selling product, monthly revenue trend, and each product's contribution to total revenue, using Pandas.

Algorithm:

1. Load sales data into a Pandas DataFrame (from CSV, or built inline here for demonstration).
2. Compute Revenue = Units × Price for every row.
3. Use `groupby()` to find total revenue per product and per month.

```
import pandas as pd

data = {
    "Date": ["2026-01-05", "2026-01-15", "2026-02-02", "2026-02-20", "2026-03-10"],
    "Product": ["Laptop", "Mouse", "Laptop", "Keyboard", "Mouse"],
    "Units": [2, 10, 1, 5, 8],
    "Price": [50000, 500, 50000, 1500, 500],
}
df = pd.DataFrame(data)
df["Date"] = pd.to_datetime(df["Date"])
df["Revenue"] = df["Units"] * df["Price"]
df["Month"] = df["Date"].dt.strftime("%Y-%m")

# Best-selling product by revenue
by_product = df.groupby("Product")["Revenue"].sum().sort_values(ascending=False)
print("Revenue by product:\n", by_product)

# Monthly revenue trend
by_month = df.groupby("Month")["Revenue"].sum()
print("\nMonthly revenue:\n", by_month)

# Contribution percentage
total_revenue = df['Revenue'].sum()
contribution = (by_product / total_revenue * 100).round(1)
print("\nRevenue contribution %:\n", contribution)
```

Output:

Revenue by product:

Laptop	150000
Mouse	9000
Keyboard	7500

Monthly revenue:

2026-01	105000
2026-02	57500
2026-03	4000

Revenue contribution %:

Laptop	90.1
Mouse	5.4
Keyboard	4.5

Explanation: `pd.to_datetime()` converts text dates into real datetime objects, enabling `.dt.strftime()` to extract a 'Year-Month' grouping key. `groupby().sum()` then implements the standard split-apply-combine analysis pattern seen in Module 18.

Project 9: Face Detection

Difficulty: Advanced

Problem Statement: Detect human faces in an image using OpenCV's pretrained Haar Cascade classifier, drawing a rectangle around each detected face.

Algorithm:

1. Load a pretrained Haar Cascade model for frontal face detection (included with OpenCV).
2. Load the target image and convert it to grayscale (detection works on grayscale).
3. Run `detectMultiScale()` to find all face regions in the image.
4. Draw a rectangle around each detected face and save/display the result.

```
# pip install opencv-python
import cv2

# Load OpenCV's built-in pretrained face detector
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml"
)

# Load an image and convert to grayscale
img = cv2.imread("group_photo.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detect faces: returns a list of (x, y, width, height) boxes
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5)
print(f"Found {len(faces)} face(s)")

# Draw a rectangle around each detected face
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)

cv2.imwrite("detected_faces.jpg", img)
print("Saved result as detected_faces.jpg")
```

Output:

```
Found 3 face(s)
Saved result as detected_faces.jpg
```

Note

Haar Cascades are an older, lightweight, CPU-friendly detection method — fast to run without a GPU, though less accurate than modern deep-learning-based face detectors. They remain an excellent, easy starting point for learning computer vision.

Explanation: `scaleFactor` controls how much the image is scaled down at each detection step (smaller = more thorough but slower), and `minNeighbors` controls how many overlapping detections are required before something is confidently called a face, reducing false positives.

Project 10: Rule-Based Chatbot

Difficulty: Advanced

Problem Statement: Build a simple chatbot that responds to common greetings and questions by matching keywords in the user's message — a foundational technique before moving to AI-powered chatbots.

Algorithm:

1. Define a dictionary mapping keywords to responses.
2. Read user input and convert it to lowercase for consistent matching.
3. Search the input for any known keyword and reply accordingly.
4. If no keyword matches, give a generic fallback response.

```
responses = {
    "hello": "Hi there! How can I help you today?",
    "hi": "Hello! What can I do for you?",
    "how are you": "I'm just a program, but I'm running well! How about you?",
    "your name": "I'm PyBot, a simple rule-based chatbot.",
    "bye": "Goodbye! Have a great day!",
    "help": "You can ask me to say hello, ask my name, or say bye to exit.",
}

def chatbot_reply(user_input):
    user_input = user_input.lower()
    for keyword, reply in responses.items():
        if keyword in user_input:
            return reply
    return "Sorry, I did not understand that. Type 'help' for options."

print("PyBot: Hello! Type 'bye' to exit.")
while True:
    user_input = input("You: ")
    reply = chatbot_reply(user_input)
    print("PyBot:", reply)
    if "bye" in user_input.lower():
        break
```

Output:

```
PyBot: Hello! Type 'bye' to exit.
You: hi there
PyBot: Hello! What can I do for you?
You: what is your name
PyBot: I'm PyBot, a simple rule-based chatbot.
You: bye
PyBot: Goodbye! Have a great day!
```

Explanation: This is called 'rule-based' because responses are hand-coded keyword matches, not learned from data. Real-world chatbots (like those built with transformer models) learn responses statistically from massive text datasets instead — but the underlying request-response loop structure is conceptually similar.

Project 11: Crop Price Prediction

Difficulty: Advanced

Problem Statement: Given historical data on rainfall and crop yield, predict a crop's market price using Linear Regression — a simplified example of the kind of model used in real agricultural analytics.

Algorithm:

1. Collect historical data: features (rainfall, yield) and target (price).
2. Split data into training and testing sets.
3. Train a Linear Regression model on the training data.
4. Use the trained model to predict prices for new, unseen input.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Historical data: [rainfall_mm, yield_tons] -> price per quintal
X = [
    [800, 20], [750, 18], [900, 25], [650, 15],
    [850, 22], [700, 16], [950, 27], [600, 13],
]
y = [2200, 2100, 2450, 1900, 2300, 1950, 2550, 1800]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1
)

model = LinearRegression()
model.fit(X_train, y_train)

accuracy = model.score(X_test, y_test)
print(f"Model R2 score on test data: {accuracy:.2f}")

# Predict price for new conditions: 820mm rainfall, 21 tons yield
new_data = [[820, 21]]
predicted_price = model.predict(new_data)
print(f"Predicted price: ₹{predicted_price[0]:.2f} per quintal")
```

Output:

Model R² score on test data: 1.00
 Predicted price: ₹2242.83 per quintal

What is R² score?

R² (R-squared) measures how well the model's predictions match actual outcomes, on a scale up to 1.0 (perfect fit). This tiny, deliberately clean illustrative dataset produces a very high score; real agricultural datasets are noisier and typically yield lower, more realistic R² scores.

Explanation: This is a simplified, small-scale illustration of a real technique. Production agricultural models use much larger datasets and often more features (soil type, region, season, market demand) plus more sophisticated algorithms — but the core workflow (collect data → split → train → predict) is exactly the same.

MORE PROJECT IDEAS FOR PRACTICE

Once you're comfortable with the projects above, try building these independently to reinforce and extend your skills:

- To-Do List App with file-based persistence (save/load tasks between runs).
- Quiz Application that reads questions from a JSON file and tracks the user's score.
- Currency Converter using a live exchange-rate API (Module 22).
- Password Generator with customisable length and character sets.
- Contact Book with search, using a SQLite database (Module 20).
- Weather Dashboard combining an API call with a Matplotlib temperature chart.
- Movie Recommendation System using simple similarity logic on a small dataset.
- Tic-Tac-Toe Game with a simple AI opponent using basic game logic.